

Resolução do Exame Modelo 3 — Paradigmas de Programação (2025/2026)

Parte 1 — Teoria (6 Valores)

Pergunta 1: Composição vs Herança (1,5 valores)

Resposta: Herança ("É UM") cria um acoplamento forte entre a subclasse e a superclasse. **Composição ("TEM UM")** envolve manter instâncias de outras classes como atributos, delegando-lhes tarefas. A composição é frequentemente preferida por manter o encapsulamento intacto, evitar a fragilidade da superclasse e permitir alterar o comportamento em tempo de execução.

```
public class Motor {
    public void ligar() { System.out.println("Vrumm!"); }
}

public class VeiculoComposto {
    private Motor motor; // Composicao: Tem Um motor
    public VeiculoComposto() { this.motor = new Motor(); }
    public void arrancar() { this.motor.ligar(); }
}
```

Pergunta 2: Tipos Enumerados (`enum`) (1,5 valores)

Resposta: Enums garantem *type safety*, centralizam valores permitidos e suportam atributos `final`, construtores privados e métodos customizados.

Pergunta 3: Classes Wrapper e Autoboxing (1,5 valores)

Resposta: Classes Wrapper (`Integer` , `Double`) envolvem tipos primitivos em objetos. **Autoboxing** é a conversão automática de primitivo para objeto; **Unboxing** é o processo inverso. O uso excessivo de autoboxing em ciclos cria milhares de objetos descartáveis na Heap, degradando o desempenho. A comparação com `==` em Wrappers compara identidades de memória e não valores numéricos, devendo usar-se sempre `equals()`.

Pergunta 4: Byte Streams vs Character Streams (1,5 valores)

Resposta: Byte Streams (`InputStream` / `OutputStream`) operam dados em bruto (8 bits), sendo usados para dados binários (imagens, zips). **Character Streams** (`Reader` / `Writer`) operam em caracteres Unicode (16 bits), sendo usados para texto legível (`.txt` , `.json`).

Parte 2 — Prática (14 Valores)

Pergunta 1a e 1b: Classe `RouteImpl` e Método `main` (5 valores)

```
public class RouteImpl implements Route {
    private Vehicle vehicle;
    private AidBox[] boxes;
    private int count;

    public RouteImpl(Vehicle vehicle) {
        this.vehicle = vehicle;
        this.boxes = new AidBox[10];
        this.count = 0;
    }

    @Override public Vehicle getVehicle() { return this.vehicle; }

    @Override
    public AidBox[] getRoute() {
        AidBox[] res = new AidBox[this.count];
        System.arraycopy(this.boxes, 0, res, 0, this.count);
        return res;
    }

    @Override
    public void addAidBox(AidBox aidBox) throws RouteException {
        if (aidBox == null) throw new RouteException("AidBox nula!");
        if (this.count >= 10) throw new RouteException("Rota cheia!");
        for (int i = 0; i < this.count; i++) {
            if (this.boxes[i].equals(aidBox)) throw new RouteException("AidBox ja existe na rota!");
        }
        this.boxes[this.count] = aidBox;
        this.count++;
    }

    @Override
    public AidBox removeAidBox(AidBox aidBox) throws RouteException {
        if (aidBox == null) throw new RouteException("AidBox nula!");
        for (int i = 0; i < this.count; i++) {
            if (this.boxes[i].equals(aidBox)) {
                AidBox removed = this.boxes[i];
                for (int j = i; j < this.count - 1; j++) {
                    this.boxes[j] = this.boxes[j + 1];
                }
                this.boxes[this.count - 1] = null;
                this.count--;
                return removed;
            }
        }
        throw new RouteException("AidBox nao encontrada!");
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || !(obj instanceof Route)) return false;
```

```

        Route other = (Route) obj;
        if (this.vehicle == null) return other.getVehicle() == null;
        return this.vehicle.equals(other.getVehicle());
    }
}

```

Pergunta 2a e 2b: Classe `CollectionManagerImpl` (9 valores)

```

public class CollectionManagerImpl implements CollectionManager {

    public double getContainerLoad(Container container) {
        if (container == null || container.getLastMeasurement() == null) return 0.0;
        return container.getLastMeasurement().getValue();
    }

    public boolean isContainerFull(Container container, double threshold) {
        if (container == null || container.getLastMeasurement() == null || container.getCapacity() == 0)
            return false;
        double occ = container.getLastMeasurement().getValue() / container.getCapacity();
        return occ > threshold;
    }

    @Override
    public double getTotalCollectedByType(IIInstitution inst, ItemType type) {
        if (inst == null || type == null) return 0.0;
        AidBox[] boxes = inst.getAidBoxes();
        if (boxes == null) return 0.0;

        double totalCollected = 0.0;

        for (int i = 0; i < boxes.length; i++) {
            AidBox box = boxes[i];
            if (box != null) {
                Container[] containers = box.getContainers();
                if (containers != null) {
                    for (int j = 0; j < containers.length; j++) {
                        Container c = containers[j];
                        if (c != null && c.getType() == type && isContainerFull(c, 0.75)) {
                            totalCollected += getContainerLoad(c);
                        }
                    }
                }
            }
        }

        return totalCollected;
    }
}

```